

Microservices Architecture

What are Microservices and Microservices Architecture?

Definizione di microservizio

I microservizi sono servizi piccoli, indipendenti e a basso accoppiamento, ciascuno focalizzato su una specifica funzionalità di business.

Le 7 caratteristiche costitutive:

1. Singola responsabilità di business

Ogni microservizio è progettato per **un'attività ben definita**, concentrata su una specifica capability aziendale. Esempi: servizio *Catalogo prodotti*, servizio *Ordini*, servizio *Pagamenti*.

2. Codebase separata e team dedicato

Ogni servizio ha **la propria codebase**, che può essere sviluppata, compresa e gestita da **un piccolo team dedicato**.

3. Deployment indipendente ★

"Un team può aggiornare e distribuire il proprio servizio senza dover ricostruire o ridistribuire l'intera applicazione, né coordinarsi in modo complesso con altri team."

Questa è la caratteristica che distingue davvero un microservizio da un modulo.

4. Comunicazione via rete con API ben definite

Comunicano tra loro **attraverso la rete**, usando API ben definite e **protocolli leggeri**: tipicamente **HTTP REST o gRPC**.

5. Incapsulamento

I **dettagli di implementazione interni** di un servizio sono **nascosti** agli altri.

6. Eterogeneità tecnologica (polyglot services)

"I servizi non devono necessariamente condividere lo stesso stack tecnologico, le stesse librerie o gli stessi framework. Un servizio potrebbe essere scritto in .NET e un altro in Java o Python, se questa è l'opzione migliore per la sua funzione specifica."

7. Database proprio ★

*"Ogni microservizio ha spesso **un proprio database o livello di persistenza che NON viene condiviso direttamente con altri servizi**. Questo rappresenta un **distacco significativo** dai modelli tradizionali in cui un singolo grande database serve l'intera applicazione."*

Definizione di architettura a microservizi (Martin Fowler)

*"Un approccio allo sviluppo di **una singola applicazione come una collezione di piccoli servizi**, ciascuno dei quali **opera nel proprio processo** e comunica tramite **meccanismi leggeri** (spesso API HTTP o gRPC). Questi servizi sono **costruiti attorno a***

capability di business e possono essere distribuiti in modo indipendente tramite meccanismi di deployment completamente automatizzati. La gestione centralizzata è minima, consentendo l'uso di diversi linguaggi di programmazione e tecnologie di data storage."

In sintesi: **scomporre un'applicazione grande e complessa in una collezione di piccoli servizi indipendenti che lavorano insieme, ciascuno gestito da un piccolo team autonomo.**

È descritto come un **approccio cloud-native**, basato sul **Single Responsibility Principle** (Robert C. Martin), applicato però **a livello di servizio** anziché di classe.

I 7 vantaggi

#	Vantaggio	Dettaglio
1	Agilità e time-to-market ★	"Spesso il fattore determinante." I team rilasciano funzionalità e fix molto più velocemente, senza impattare l'intera applicazione né attendere altri team
2	Scalabilità granulare ★	"Se il servizio di raccomandazione richiede elevate risorse CPU, si scala solo quel servizio, non l'intera applicazione " → utilizzo più efficiente delle risorse e risparmio sui costi
3	Team piccoli e autonomi	Team cross-funzionali che gestiscono i propri servizi end-to-end . Aumenta produttività e innovazione
4	Isolamento dei guasti	"Se un servizio si blocca, non significa necessariamente che l'intera applicazione vada in crash " — a condizione che il sistema sia progettato per gestire i guasti con grazia
5	Diversità tecnologica	"Lo strumento giusto per il compito giusto." Ogni team sceglie linguaggio, framework e data store più adatti
6	Codebase piccola e focalizzata	Più facile da comprendere, modificare e mantenere
7	Evoluzione indipendente dei dati	Ogni servizio gestisce il proprio schema → "gli aggiornamenti dei dati in un servizio non influiscono direttamente sugli altri"

Le 6 sfide

1. Complessità complessiva del sistema ★

"Sebbene i singoli servizi siano più semplici, il sistema nel suo insieme — con molti microservizi che interagiscono — diventa un complesso sistema distribuito. Gestire queste parti mobili è una sfida significativa."

Questo è il trade-off fondamentale: **sposti la complessità dal codice all'infrastruttura.**

2. Latenza e affidabilità della rete ★

*"A differenza di un monolite dove le chiamate sono **in-process**, la comunicazione tra microservizi avviene **attraverso la rete**, che è **intrinsecamente meno affidabile e introduce latenza**."*

Una chiamata a metodo passa da ~nanosecondi a ~millisecondi, e **può fallire.**

3. Coerenza dei dati distribuiti ★

*"Questa è **una delle sfide più difficili**. Quando le transazioni di business si estendono su più servizi, servono **pattern complessi come le Saga**. L'**eventual consistency diventa il modello comune**."*

È il prezzo diretto di aver perso le transazioni ACID del monolite (Lezione 3). → risolto nella

4. Complessità dei test

Testare un singolo servizio è semplice. **Test di integrazione ed end-to-end su più servizi** diventano significativamente più complessi e critici.

5. Complessità di deployment e costi operativi

*"Distribuire e gestire **decine o centinaia di servizi** richiede un **elevato grado di automazione**: piattaforme robuste, **pipeline CI/CD, container e orchestrazione**. È un investimento significativo."*

6. Debug e monitoring distribuiti

*"Quando una richiesta fallisce o è lenta, **tracciarne il problema attraverso più servizi** per trovare la causa radice è molto più difficile. **Logging centralizzato, distributed tracing e monitoring completo sono essenziali** — ma aggiungono complessità."*

★ La conclusione onesta

*"Passare ai microservizi **non è una soluzione miracolosa (silver bullet)**. È un **trade-off**. Si rinuncia ad alcuni vantaggi del monolite in cambio di una **maggiore complessità operativa**, ottenendo **agilità e scalabilità indipendente**."*

When to Use / When NOT to Use Microservices Architecture

Il principio guida

*"Non adottate i microservizi solo perché sono di moda. Architetti e sviluppatori dovrebbero concentrarsi sui **problemi di business** che stanno cercando di risolvere, e chiedersi se i microservizi li risolvano effettivamente."*

✓ QUANDO usarli — i 5 casi

1. La scalabilità indipendente di un componente è critica

Se parti diverse dell'applicazione hanno **requisiti di scalabilità significativamente diversi**. (Esattamente il limite identificato nella Lezione 4.)

2. Serve alta agilità con deployment rapidi e indipendenti

Se più team devono rilasciare **frequentemente e indipendentemente** funzionalità per le proprie aree di business. *"Ogni servizio può avere la propria pipeline di deployment."*

3. La diversità tecnologica è un vantaggio strategico

Esempio citato: **Python** per servizi di raccomandazione basati su machine learning, insieme a **microservizi .NET** per le applicazioni core.

4. Un monolite grande e complesso sta causando problemi reali

Che impattano velocità di sviluppo, produttività del team e affidabilità — e questi problemi coincidono con ciò che i microservizi risolvono.

5. L'organizzazione è (o vuole essere) strutturata in piccoli team autonomi

Ciascuno con piena ownership di specifiche capability di business.

★ La raccomandazione di Sam Newman e Martin Fowler

"Una prassi comune, spesso raccomandata da esperti come Sam Newman e Martin Fowler, è l'approccio 'Monolith First': partire da un'applicazione ben strutturata e monolitica — come abbiamo fatto — e poi, se emergono esigenze specifiche, estrarre gradualmente e strategicamente i singoli moduli in microservizi distinti."

"Questo è esattamente il percorso che simuliamo in questo corso."

È la validazione retroattiva di tutte le lezioni 2, 3 e 4: non erano una perdita di tempo, erano **il modo corretto di arrivare qui**.

✗ QUANDO NON usarli — i 6 casi

1. Applicazione piccola o startup in fase iniziale

"Se l'applicazione è piccola, il mercato di riferimento è ancora instabile, o il team è ridotto, i costi della complessità superano di gran lunga qualsiasi potenziale beneficio. Un monolite ben strutturato è sempre un punto di partenza migliore."

E un'osservazione acuta:

"Le startup spesso fanno pivot cruciali, e modificare i confini dei servizi in un'architettura a microservizi può essere estremamente costoso."

In un monolite sposti una classe. Con i microservizi devi rinegoziare API, migrare database, coordinare deployment.

2. Manca una solida cultura DevOps e capacità di automazione

"I microservizi richiedono un elevato grado di automazione per provisioning, monitoring e CI/CD. Gestire manualmente un insieme di microservizi è insostenibile e porterà inevitabilmente a problemi."

3. Il team non ha esperienza con i sistemi distribuiti

Inaffidabilità della rete, latenza, eventual consistency, distributed tracing. *"La curva di apprendimento sarà ripida e probabilmente si tradurrà in servizi mal progettati."*

4. Manca un chiaro incentivo di business

"Se l'applicazione ha prestazioni ragionevoli e possiamo rilasciare funzionalità a un ritmo accettabile, il monolite è più adatto alle nostre esigenze attuali rispetto all'enorme aumento di complessità. Non creiamo microservizi solo per il gusto di crearli."

5. Rischio di creare un Distributed Monolith (vedi sotto)

6. Manca l'infrastruttura di monitoring

"Se non siete disposti a impegnarvi su monitoring sofisticato, logging centralizzato e distributed tracing, sarete alla cieca."

★ L'anti-pattern: il Distributed Monolith

Definizione: un sistema in cui l'applicazione è stata divisa in servizi separati, ma questi servizi sono rimasti strettamente accoppiati tra loro.

Come si manifesta:

- chiamate sincrone eccessive e non coordinate tra servizi
- oppure, peggio, **condivisione di database** in modo che impedisce lo sviluppo e il deployment indipendenti

Perché è il peggiore degli scenari possibili:

*"Un monolite distribuito presenta **tutte le complessità dei microservizi** — chiamate di rete, deployment multipli, tracing — **senza offrirne alcun vantaggio** come l'indipendenza dei team o la flessibilità. **Il debug di servizi distribuiti strettamente accoppiati è un incubo.**"*

Hai pagato il prezzo **due volte** senza comprare nulla.

La causa radice:

*"I microservizi si basano sull'indipendenza. **Il loose coupling è fondamentale**, quindi è molto importante **segmentare i servizi in base ai confini corretti.**"*

→ È proprio il problema che apre la Lezione 6.

L'anti-pattern del database condiviso

*"Un anti-pattern specifico da evitare è **un database condiviso tra servizi nominalmente indipendenti**. Sebbene tecnicamente possibile, se i servizi condividono direttamente tabelle o schemi in modo da creare forti dipendenze, **si perde la capacità di deployment e sviluppo indipendente** — i vantaggi chiave dei microservizi. Ogni servizio deve avere i propri dati."*

📌 Il caso Amazon

*"C'è il case study di Amazon, che descrive **la riprogettazione della loro architettura da microservizi distribuiti a un'architettura monolitica**. In questo modo sono riusciti a **ridurre i costi del 90%.**"*

(Si riferisce al noto articolo del team **Amazon Prime Video** del 2023, sul loro servizio di monitoraggio audio/video: lo avevano costruito con funzioni serverless e step function distribuite, e lo hanno ricondotto a un singolo processo monolitico ottenendo circa il 90% di riduzione dei costi. Nota importante: riguarda **uno specifico servizio interno**, non Amazon nel suo complesso — che resta uno dei più grandi utilizzatori di microservizi al mondo. Il punto della lezione resta comunque valido: **l'architettura giusta dipende dal caso d'uso, non dalla moda.**)

Le 5 domande da porsi prima di migrare

1. **Il monolite è davvero un collo di bottiglia** che ci impedisce di raggiungere obiettivi di business critici (rilascio più veloce, scalabilità indipendente di un componente resource-intensive)?
 2. Possiamo invece **migliorare o ottimizzare i moduli esistenti**? *"A volte migliorare l'architettura interna o fare performance tuning può posticipare la necessità di microservizi."*
 3. **Abbiamo parti del sistema chiaramente definite con requisiti distinti** di scalabilità, disponibilità o stack tecnologico, tali da giustificarne la separazione **ora**?
 4. **Il nostro team ha le competenze e la maturità** per DevOps e sistemi distribuiti?
 5. ★ **I benefici attesi superano significativamente l'innegabile aumento di complessità** architetturale e operativa?
-

The Database-per-Service Pattern & Polyglot Persistence

Definizione: Database-per-Service

Ogni microservizio possiede e gestisce il proprio database. Quel database è dedicato a quel servizio e contiene solo dati rilevanti per la sua specifica capability di business.

La regola ferrea:

"Nessun altro microservizio è autorizzato ad accedere direttamente al database di un altro servizio. Non esiste uno schema condiviso, e non sono consentite letture o scritture dirette tra servizi a livello di database."

Come si accede ai dati altrui:

*"Se un servizio necessita di dati posseduti da un altro servizio, **deve ottenerli chiamando l'API esposta dal servizio proprietario.**"*

Esempio esplicito:

*"Se il servizio **Ordering** necessita dei dettagli di un prodotto, **chiama l'API del servizio Catalog.** Non interroga direttamente il database del servizio Catalog."*

Questo pattern introduce **un solido incapsulamento dei dati**, essenziale per la vera indipendenza dei servizi.

I 6 vantaggi

Vantaggio	Dettaglio
Loose coupling e indipendenza ★	Non condividendo i database, i servizi possono essere sviluppati, distribuiti e scalati indipendentemente . Le modifiche al modello dati di un servizio hanno impatto diretto minimo sugli altri
Evoluzione autonoma dello schema	<i>"Un team può modificare lo schema del proprio database senza coordinarsi con altri team, purché l'API pubblica resti compatibile. Questo accelera significativamente lo sviluppo"</i>
Scalabilità	Servizi diversi hanno esigenze e profili di carico diversi → ogni database

Vantaggio	Dettaglio
indipendente degli storage	scala indipendentemente
Flessibilità tecnologica	Si sceglie la tecnologia di database più adatta al modello dati e al carico di lavoro di ciascun servizio
Isolamento dei guasti	<i>"Se il database di un microservizio si guasta, l'impatto si limita principalmente a quel servizio. Gli altri continuano a funzionare"</i>
Proprietà dei dati chiara	Rafforza la definizione dei confini tra capability di business

Definizione: Polyglot Persistence

Usare tecnologie di data storage diverse per microservizi diversi, anziché essere costretti a un singolo database "one-size-fits-all" per l'intera applicazione.

È **abilitata direttamente** dal pattern Database-per-Service: se ogni servizio possiede il proprio database, ogni servizio può scegliere *quale* database.

Gli esempi concreti dell'e-commerce

Microservizio	Caratteristiche dei dati	Database scelto	Perché
Catalog	Informazioni prodotto complesse e annidate, alto volume di letture	MongoDB / Couchbase (NoSQL documentale)	<i>"Ottimi per schemi flessibili e per scalare le letture"</i>
Ordering	Dati strutturati e transazionali, relazioni tra ordini, clienti, articoli	PostgreSQL / MySQL (relazionale)	Proprietà ACID , integrità referenziale
Basket	Letture e scritture velocissime di dati temporanei di sessione	Redis (key-value in memoria)	Latenza sub-millisecondo; i dati sono effimeri
Search	Ricerca full-text, faceting, ranking	Elasticsearch / OpenSearch	Motore di ricerca specializzato
<i>"Questo principio promuove l'uso del database più adatto alla funzione più appropriata."</i>			

Le 2 sfide che il pattern introduce

1. Aggregazione dei dati cross-service ★

*"Se devo creare un report che **combina informazioni sull'ordine e dettagli del prodotto**, come faccio quando i dati risiedono in **due database separati appartenenti a due microservizi diversi**?"*

Nel monolite era una JOIN. Ora non puoi fare join tra MongoDB e PostgreSQL su macchine diverse.

→ risolto nella **Lezione 11** (CQRS, materialized views, API Composition).

2. Transazioni distribuite e coerenza ★

*"Cosa succede se **un singolo processo di business** — come l'inserimento di un ordine — **coinvolge aggiornamenti su più microservizi**, ognuno con il proprio database? Come si garantisce la coerenza dei dati?"*

Design: E-Shop Microservices Architecture

Perché stiamo migrando (riepilogo)

- Il business è in forte espansione, con **pressione da prodotto, vendite e pagamenti** per aggiungere funzionalità immediatamente
- Serve **innovare e sperimentare rapidamente**
- I team vogliono **rilasciare non appena pronti**, senza essere vincolati a una rigida pianificazione delle release
- Serve **scalabilità flessibile e indipendente** per gestire picchi come il **Black Friday** e milioni di ordini

*"Sappiamo che **non si tratta solo di un cambiamento tecnologico**: spesso va di pari passo con **cambiamenti organizzativi** volti a una maggiore autonomia dei team."*

*(È il riferimento implicito alla **Legge di Conway**: l'architettura di un sistema riflette la struttura comunicativa dell'organizzazione che lo produce.)*

Toolkit aggiornato

Requisiti funzionali: invariati.

Nuovi requisiti non funzionali — 6:

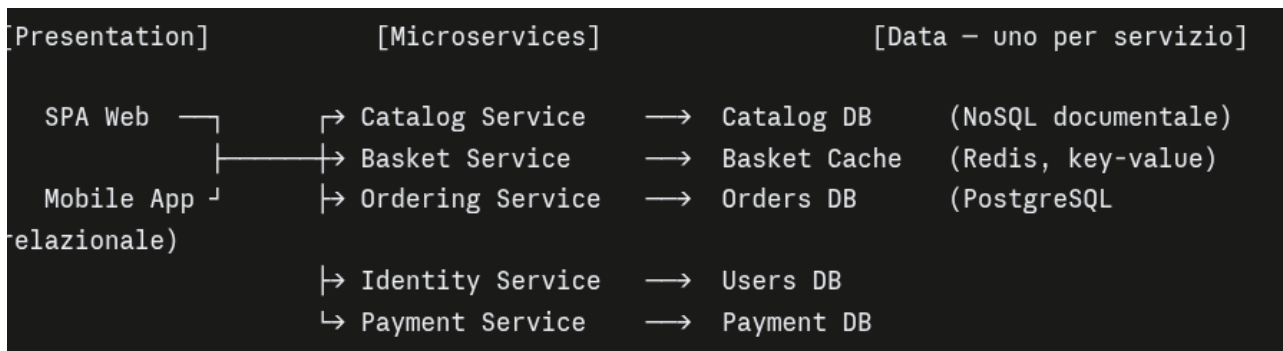
1. **Alta scalabilità granulare e indipendente** per ogni servizio
2. **Alta disponibilità** sia per i singoli servizi sia per il sistema intero
3. **Capacità di gestire milioni** di utenti e richieste simultanee
4. **Deployment indipendente** per ciascun servizio ★
5. **Indipendenza tecnologica**
6. **Isolamento dei dati** per ciascun servizio ★ + **resilienza e isolamento dei guasti**

Nuovi pattern e principi nel toolkit:

- **Single Responsibility Principle applicato a livello di servizio** — *"ogni microservizio deve avere una singola responsabilità chiaramente definita"*
- **Database-per-Service**
- **Polyglot Persistence**

*"Questo rappresenta un **cambio di prospettiva**, incentrato sulla scomposizione del problema in parti più piccole e indipendenti."*

Il design



I microservizi progettati

Servizio	Responsabilità	Database proposto
Catalog	Tutte le informazioni sui prodotti	NoSQL documentale — <i>"ottimizzato per schemi flessibili e alta velocità di lettura"</i>
Basket	Carrelli attivi degli utenti	Redis key-value — <i>"dedicato alla cache del carrello"</i>
Ordering	Creazione ordini, cronologia, logica correlata	PostgreSQL — <i>"per gestire i dati transazionali"</i>
Identity	Account utente, autenticazione, autorizzazione	Database utenti dedicato
Payment	Elaborazione pagamenti	Data store dedicato

I 4 principi del design

1. **Servizi piccoli e indipendenti** che comunicano tramite **API ben definite** (RESTful o gRPC)
2. **Organizzati attorno a capability di business e bounded context**
3. **Database-per-Service** — ogni servizio ha i propri dati e la propria tecnologia
4. **Accesso ai dati altrui solo via API:**

*"Un microservizio **non** interrogherà direttamente il database dei prodotti, ma chiamerà l'API del microservizio Catalog se necessita di dettagli sul prodotto."*

Scelte tecnologiche

Presentation: qualsiasi framework moderno (Angular, React, Vue). Il corso usa **Blazor**.

Backend — l'idea del polyglot in pratica:

*"Il microservizio **Catalog**, se richiede alta scalabilità e schema flessibile, potrebbe essere sviluppato in **Node.js con MongoDB**. Un futuro servizio **Payment** potrebbe essere implementato in **Java**, per specifiche integrazioni con librerie enterprise o di sicurezza. Un servizio potrebbe usare **Python** se il team ha quelle competenze."*

★ Il chiarimento importante — poliglottismo non significa caos:

*"Questo **non** significa che ogni microservizio debba usare uno stack diverso. Le organizzazioni spesso adottano **uno stack di base** — .NET o Java — **ma consentono deviazioni per servizi specifici con requisiti unici**. Ad esempio, se hai requisiti di machine learning, usi **Python** per quel servizio, mentre il resto è in .NET o Java."*

Questa è la posizione matura: **stack di default + eccezioni motivate**. Avere 12 linguaggi in 12 servizi è un incubo di manutenzione.

Code: E-Shop Microservices Architecture

Cartella 6-eshop-microservices-first.

★ Il cambiamento nell'AppHost

"Come potete vedere, l'AppHost è cambiato significativamente."

Prima (modular monolith): 1 progetto backend, 1 database.

Ora: N progetti backend, N database.

I backing services — uno per servizio

```
// Catalog → PostgreSQL
var catalogDb = builder.AddPostgres("postgres")
    .AddDatabase("catalogdb");

// Basket → Redis
var cache = builder.AddRedis("cache");

// Ordering → SQL Server
var orderDb = builder.AddSqlServer("sqlserver")
    .AddDatabase("orderdb");
```

*"Come potete vedere, **questo dimostra la polyglot persistence**: servizi diversi possono usare **tecnologie di database diverse**, più adatte alle loro esigenze, e **ogni servizio ha la propria istanza di database dedicata**."*

Nota la scelta concreta del corso: **PostgreSQL** per Catalog, **Redis** per Basket, **SQL Server** per Ordering. Tre motori diversi in un'unica applicazione.

I progetti — ora sono veri servizi

*"Ora abbiamo **definizioni di progetto separate** per Catalog, Basket e Ordering, e **ognuno è un progetto ASP.NET Core Web API distinto** — **non semplicemente una class library**."*

Questa è LA differenza rispetto alla Lezione 3. Nel modular monolith i moduli erano *class library* caricate in un unico processo. Ora sono **applicazioni web autonome, con un proprio processo, una propria porta, un proprio ciclo di vita**.

```
var catalog = builder.AddProject<Catalog>("catalog").WithReference(catalogDb);
var basket = builder.AddProject<Basket>("basket").WithReference(cache)
    .WithReference(ordering); // ⚠️
var ordering = builder.AddProject<Ordering>("ordering").WithReference(orderDb);
```

```
builder.AddProject<WebApp>("webapp")
    .WithReference(catalog)
    .WithReference(basket)
    .WithReference(ordering);
```

Il debito tecnico dichiarato: la chiamata sincrona Basket → Ordering

*"In questa prima fase, ho mantenuto **una singola connessione sincrona diretta**: il microservizio **Basket** chiamerà il microservizio **Ordering** durante il checkout. Per questo aggiungiamo un riferimento da Basket a Ordering."*

"Idealmente, la comunicazione tra Basket e Ordering dovrebbe essere asincrona, come vedremo nelle sezioni successive."

Questo è didatticamente importante: il corso **introduce deliberatamente un accoppiamento sincrono** per poi mostrare nella **Lezione 9** perché è un problema e come risolverlo con la messaggistica asincrona. È un mini-ciclo Problem → Learn → Design impostato in anticipo.

La struttura della soluzione

```
eshop-microservices/
├─ AppHost/                ← orchestrazione
├─ ServiceDefaults/
├─ Frontend/
│   └─ WebApp/             ← SPA + ApiClients
└─ Services/               ← I MICROSERVIZI
    ├─ Catalog/            ← progetto ASP.NET Core Web API
    ├─ Basket/             ← progetto ASP.NET Core Web API
    └─ Ordering/           ← progetto ASP.NET Core Web API
```

Struttura interna di un microservizio (es. Catalog) — identica a quella dei moduli della Lezione 3:

```
Catalog/
├─ Models/                → entità
├─ Data/                  → DbContext, accesso a PostgreSQL
├─ Services/              → ProductService (logica di business)
└─ Endpoints/             → ProductEndpoints (espone le API)
```

*Questa struttura segue **lo stesso pattern per ogni microservizio internamente**."*

È di nuovo la distinzione **architettura interna vs esterna** della Lezione 2: cambia l'architettura esterna, l'interna resta N-layer.

Lato client: N HttpClient invece di 1

In WebApp/Program.cs ora ci sono **tre** configurazioni di HttpClient:

```
builder.Services.AddHttpClient<CatalogApiClient>(c =>
```

```

c.BaseAddress = new("https+http://catalog");
builder.Services.AddHttpClient<BasketApiClient>(c =>
    c.BaseAddress = new("https+http://basket"));
builder.Services.AddHttpClient<OrderingApiClient>(c =>
    c.BaseAddress = new("https+http://ordering"));

```

Ognuno punta a un **nome logico di servizio** risolto dal **service discovery** di Aspire.

In WebApp/ApiClients/ ci sono le classi client con i metodi corrispondenti:

CatalogApiClient.GetProducts(), GetProductById(),
BasketApiClient.AddToCart(), Checkout(), ecc.

Il flusso di un click: l'utente clicca "Aggiungi al carrello" → BasketApiClient.AddToCart() → **HTTP POST** all'endpoint del microservizio Basket → BasketEndpoints → BasketService.AddToCart() → Redis.

Nota il cambiamento concettuale rispetto alla Lezione 3: lì la stessa operazione era una chiamata a metodo in-process. Ora è **una richiesta HTTP sulla rete**, con serializzazione JSON, latenza e possibilità di fallimento.

Cosa si osserva eseguendo

La dashboard Aspire mostra ora:

- **Backing services:** PostgreSQL (catalogdb) + pgAdmin, Redis (basket cache) + Redis Insight, SQL Server (orderdb)
- **Microservizi:** catalog, basket, ordering
- **Frontend:** webapp

Nel **resource graph** si vede chiaramente:

- WebApp → Catalog, Basket, Ordering
- **Basket** → **Ordering** (la chiamata sincrona del checkout)
- Catalog → PostgreSQL, Basket → Redis, Ordering → SQL Server

*"Questa è la nostra rappresentazione visiva dell'**architettura a microservizi distribuiti**."*

★ I casi d'uso end-to-end nel tracing

Punto fondamentale:

*"Dal punto di vista dell'utente, l'applicazione si comporta in modo identico. Tuttavia, le operazioni sono ora gestite da **microservizi distribuiti**."*

1. Lista prodotti — traccia:

WebApp → HTTP GET → Catalog Service → query SQL → PostgreSQL

2. Aggiungi al carrello — traccia:

WebApp → HTTP GET → Basket Service → legge da **Redis** (recupera carrello utente)
→ HTTP POST → Basket Service → scrive su Redis

Verificabile in **Redis Insight**: si vede la chiave del carrello con dentro i due articoli e lo username.

3. Checkout — la traccia più interessante ★

WebApp

```
└→ HTTP POST → Basket Service (/checkout)
    └→ legge il carrello da Redis (recupera items e prezzo totale)
    └→ HTTP POST → Ordering Service ← CHIAMATA SINCRONA TRA SERVIZI
        └→ INSERT nella tabella orders (SQL Server)
└→ elimina il carrello da Redis
```

*"La traccia parte dalla WebApp, invia una **HTTP POST** all'endpoint del microservizio Basket, che esegue diverse operazioni: prima **recupera i dati dalla cache Redis**, poi **invia una HTTP POST all'endpoint di Ordering** per creare un nuovo record d'ordine in SQL Server, e infine **elimina il carrello da Redis**."*

Verifiche finali:

- **Redis Insight**: la cache non contiene più chiavi ✓
- **SQL Server Object Explorer**: il nuovo record è nella tabella Orders ✓ (la connection string si copia dalla dashboard Aspire, sezione Ordering → variabili d'ambiente)

Osservazione da tenere a mente: questa traccia di checkout è **fragile**. Se Ordering è temporaneamente down, l'intero checkout fallisce e l'utente perde l'operazione, pur avendo il carrello ancora valido. È esattamente il problema che motiverà le Lezioni 9 (async) e 15 (resilienza).

Problem: Validating Our Service Boundaries — How to Decompose Microservices

EVAL → PROBLEM della Lezione 6.

La roadmap delle considerazioni verticali

*"D'ora in poi esamineremo i microservizi da diverse prospettive. Le considerazioni principali sono: **decomposizione e confini dei servizi, comunicazione, gestione dei dati**. Poi esploreremo **gestione delle transazioni e resilienza**."*

Questa è la mappa delle Lezioni 6–15.

Cosa abbiamo ottenuto

*"La nostra applicazione ha ora un backend composto da **microservizi separati** — Catalog, Basket, Ordering, Identity. **Questo è un passo avanti significativo** rispetto al monolite con cui abbiamo lavorato finora."*

★ La domanda critica

"Ma questi sono i servizi giusti, e i loro confini sono ideali?"

Il ragionamento che segue è il più importante della sezione:

*"Se i confini dei nostri servizi **non sono ben definiti**, potremmo ritrovarci con **un monolite distribuito** in cui i servizi **si sovrappongono, si duplicano, o non sono allineati** al funzionamento della nostra azienda o dei nostri team."*

Nota il punto: **avere microservizi non garantisce i benefici dei microservizi**. Li ottieni solo se i **confini** sono corretti.

Le domande specifiche da risolvere

1. **Questi servizi sono veramente indipendenti?**
2. **Incarnano un insieme coerente di responsabilità?** I loro confini sono allineati alla struttura del business e dei team?
3. **Le dimensioni sono appropriate?** ★

*"Non troppo grandi — mini-monoliti — e **non troppo piccoli** — **nanoservizi** che creano un'eccessiva comunicazione tra microservizi."*

4. **Questa decomposizione ci permette davvero di raggiungere** l'indipendenza, la scalabilità e il deployment di cui abbiamo bisogno?
5. **Un esempio concreto di dubbio:**

*"Il nostro servizio **Ordering** è **troppo generico**? **L'elaborazione dei pagamenti dovrebbe essere un microservizio distinto**, per scalabilità e sicurezza specializzate, anziché essere integrata dentro Ordering?"*

I 4 sintomi di una decomposizione sbagliata

Sintomo	Descrizione
Servizi ancora troppo accoppiati	Richiedono deployment coordinati — hai perso il beneficio principale
Comunicazione eccessiva ("chatty")	Troppe chiamate tra servizi → problemi di prestazioni e cascate di latenza
Team che non riescono a gestire i servizi indipendentemente	I confini non sono allineati all'organizzazione
Discrepanza nelle unità di scalabilità	<i>"Stiamo ancora scalando insieme parti che non hanno bisogno di essere scalate insieme"</i>

La soluzione

*"Come soluzione, dobbiamo conoscere e approfondire **i pattern di decomposizione dei microservizi e le strategie di decomposizione**."*
